

Recap: GPT-3 learns in-context

Paper: <https://arxiv.org/abs/2005.14165>

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 cheese => ..... ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← examples
3 peppermint => menthe poivrée ←
4 plush girafe => girafe peluche ←
5 cheese => ..... ← prompt
```

Part 1/3: Ancient History of conversation systems



Ye olde chat bots: ELIZA

Source: ELIZA by Joseph Weizenbaum, 1967

Welcome to

EEEEEE	LL	IIII	ZZZZZZ	AAAAA
EE	LL	II	ZZ	AA AA
EEEEE	LL	II	ZZZ	AAAAAAA
EE	LL	II	ZZ	AA AA
EEEEEE	LLLLLL	IIII	ZZZZZZ	AA AA

Eliza is a mock Rogerian psychotherapist.

The original program was described by Joseph Weizenbaum in 1966.

This implementation by Norbert Landsteiner 2005.

ELIZA: Is something troubling you ?

YOU: Men are all alike.

ELIZA: What is the connection, do you suppose ?

YOU: They're always bugging us about something or other.

ELIZA: Can you think of a specific example ?

YOU: Well, my boyfriend made me come here.

ELIZA: Is it important to you that your boyfriend made you come here ?

YOU: He says I'm depressed much of the time.

ELIZA: I am sorry to hear that you are depressed.

YOU: It's true. I am unhappy.

ELIZA: Can you explain what made you unhappy ?

YOU:

Ye olde chat bots: ELIZA

Source: ELIZA by Joseph Weizenbaum, 1967

Welcome to

```
EEEEEE LL      IIII  ZZZZZZ  AAAAA
EE      LL      II    ZZ      AA   AA
EEEEEE LL      II    ZZZ      AAAAAA
EE      LL      II    ZZ      AA   AA
EEEEEE LLLLLL IIII  ZZZZZZ  AA   AA
```

Eliza is
The orig
This imp

ELIZA was manually programmed to ask questions about user's previous message

1966.

People who use the chatbot report that ELIZA feels like *she* is almost human (see: ELIZA effect)

Note: there were others, e.g. PARRY (1971)

ELIZA: Is
YOU: Men
ELIZA: Wha
YOU: The
ELIZA: Can
YOU: Wel

ELIZA: Is it important to you that your boyfriend made you come here ?
YOU: He says I'm depressed much of the time.
ELIZA: I am sorry to hear that you are depressed.
YOU: It's true. I am unhappy.
ELIZA: Can you explain what made you unhappy ?
YOU:

GPT

	<u>GPT-1</u> (2018)	<u>GPT-2</u> (2019)	<u>GPT-3</u> (2020)
Number of parameters	117 million	1.5 billion	175 billion
Training data	5 GB	40 GB	45 TB (i.e., 45 000 GB)

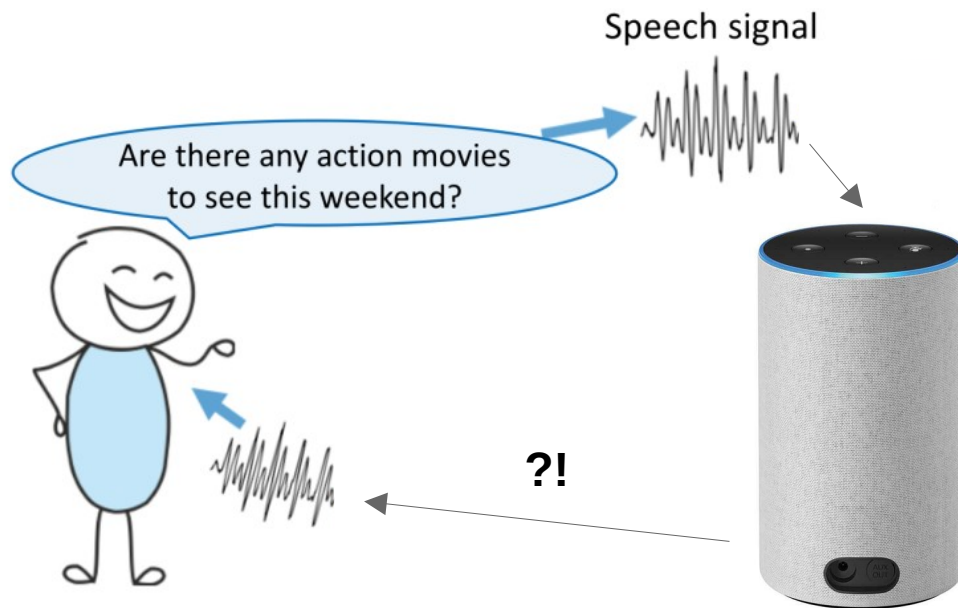
The entirety of English
Wikipedia constitutes just
0.6% of GPT-3 training data

Part 2/3: goal oriented chat bots and voice assistants



Goal oriented chat bots

System design: let's split “chat bot” into smaller problems



Q: how would you design it?

e.g. a bank tech support bot

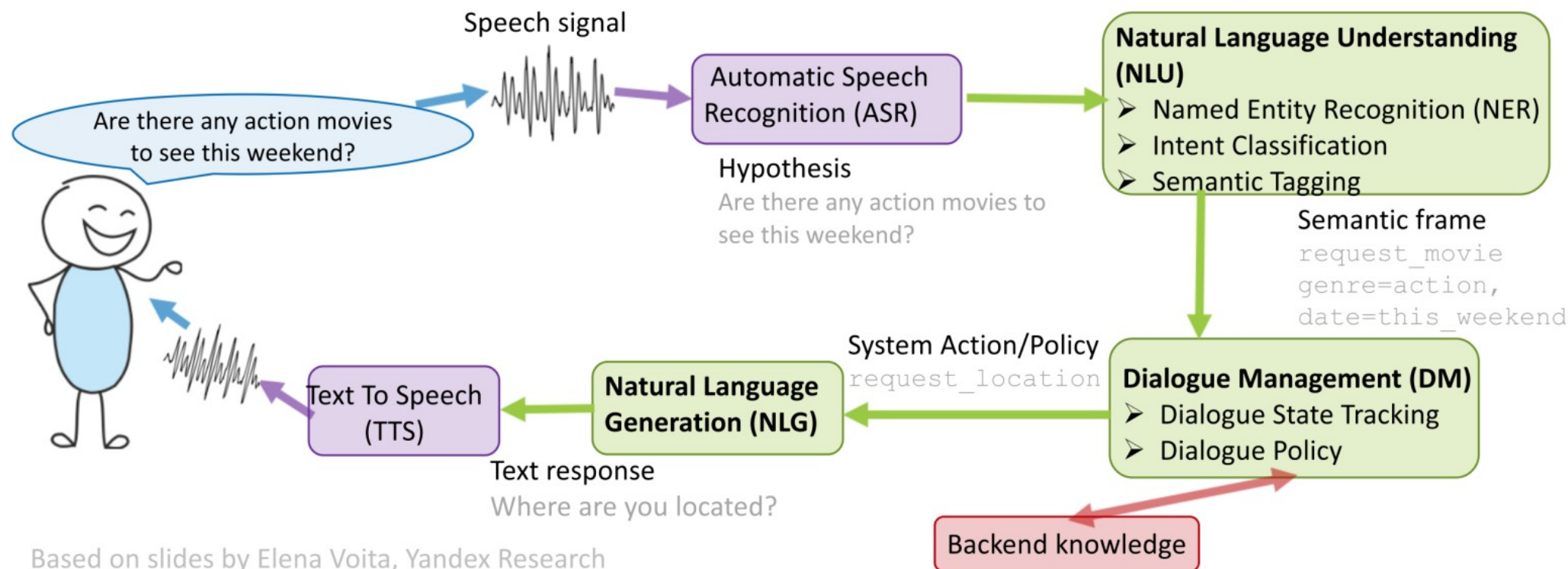
Goal oriented chat bots

System design: let's split “chat bot” into smaller problems

Speech processing: see YSDA speech course

Business knowledge: rules for banking / psychology / ...

Text processing: this lecture



Goal oriented chat bots

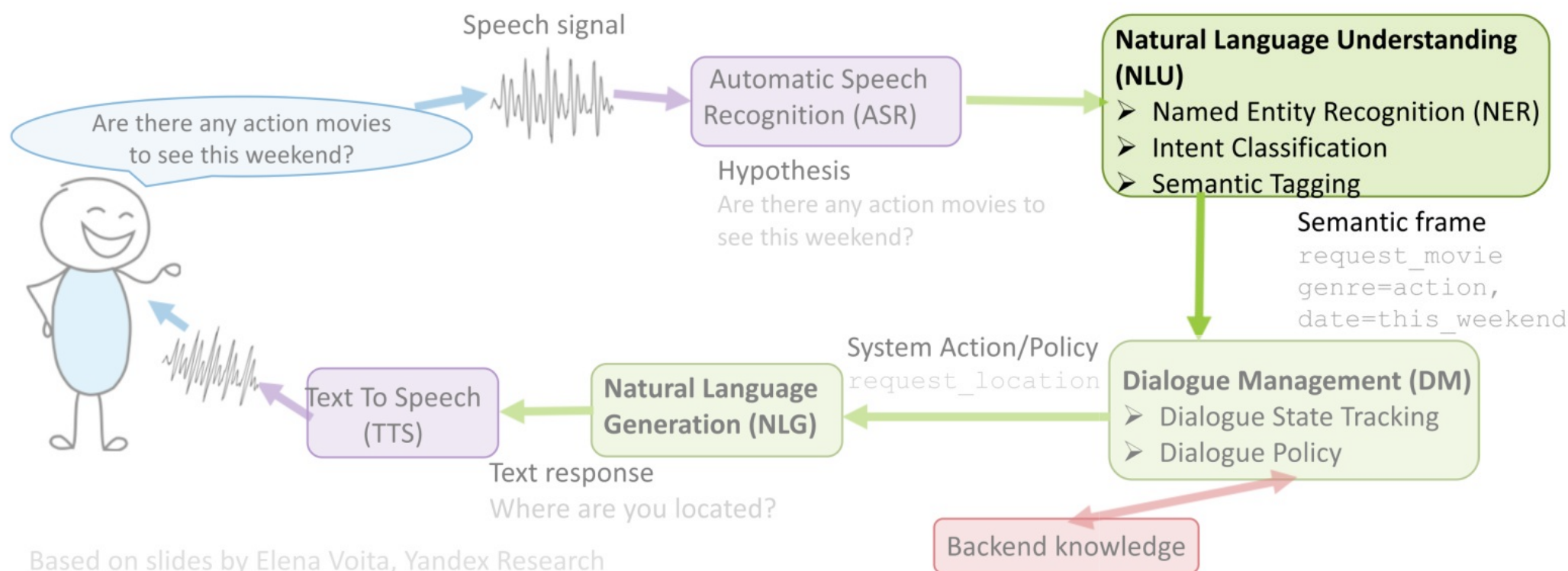
System design: let's split "chat bot" into smaller problems

Speech processing: see YSDA speech course

Business knowledge: rules for banking / psychology / ...

Text processing: this lecture

look into this one



Named Entity Recognition

Why: extract keywords from user's message, use them , e.g.

- for web search, when checking a fact
- for map look-up (“where can I buy pizza in Taganrog?”)
- to play music / video (“play Eminem’s latest song”)

Q: how do we solve this?

When **Sebastian Thrun PERSON** started working on self - driving cars at **Google ORG** in **2007 DATE** , few people outside of the company took him seriously . “ I can tell you very senior CEOs of major **American NORP** car companies would shake my hand and turn away because I was n’t worth talking to , ” said **Thrun PERSON** , in an interview with **Recode ORG** earlier this week **DATED** .

Image credit: nanonets.com

Named Entity Recognition

Why: extract keywords from user's message, use them , e.g.

- for web search, when checking a fact
- for map look-up ("where can I buy pizza in Taganrog?")
- to play music / video ("play Eminem's latest song")

Q: how do we solve this?

A: fine-tune a BERT-like model
for token classification
See week 5 for details :)



```
1 text = "<YOUR TEXT HERE>"
2 pipeline = transformers.pipeline("ner", "dslim/bert-base-NER")
3 print("Found named entities", pipeline(text))
```

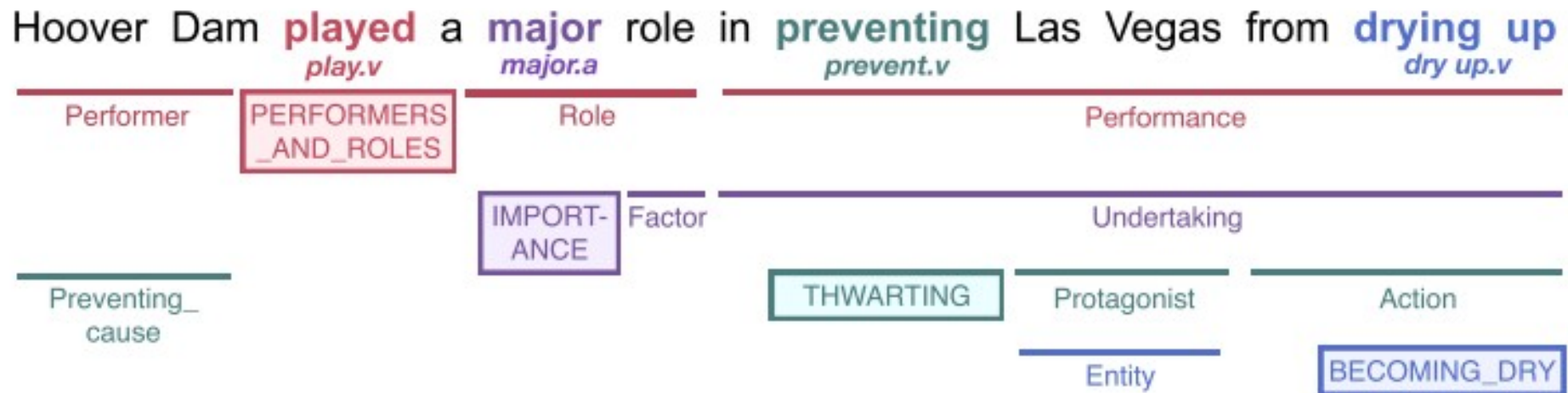


Found named entities [{'entity': 'B-LOC', 'score': 0.8838524, 'index': 30, 'wo



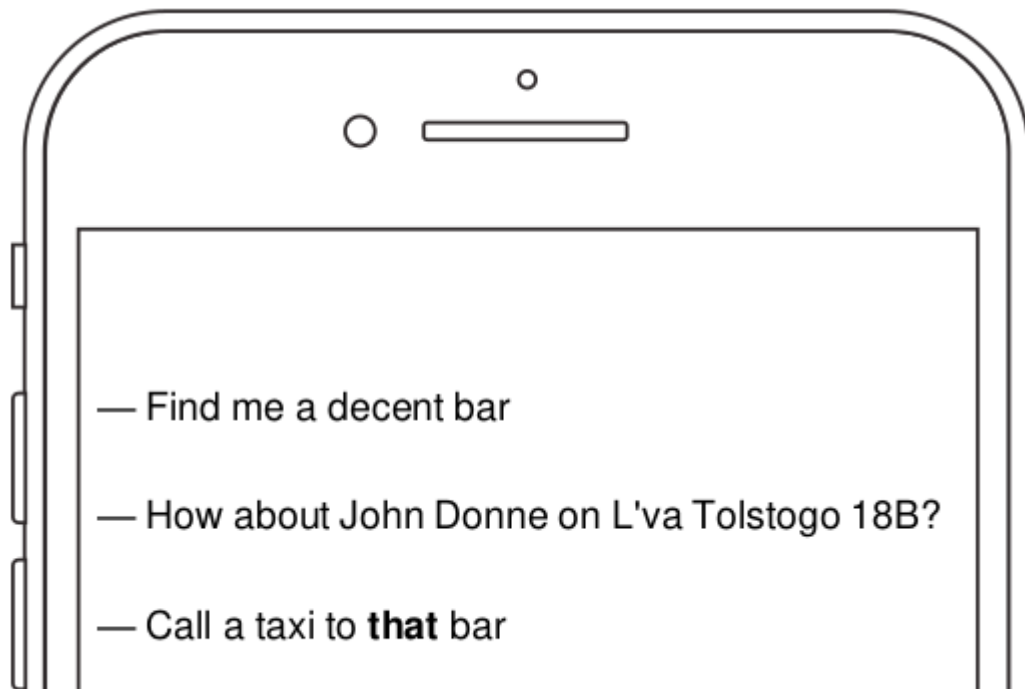
More NLU problems

- **Named Entity Recognition:** see previous slide
- **Semantic parsing:** to figure out what user wants from you



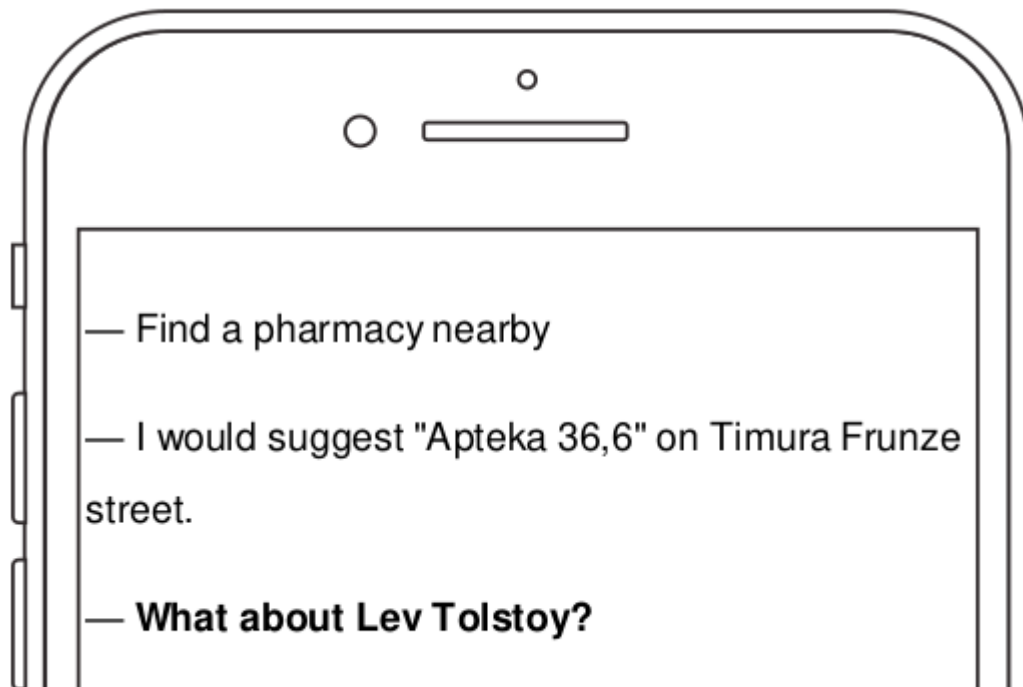
More NLU problems

- **Named Entity Recognition:** see previous slide
- **Semantic parsing:** to figure out what user wants from you
- **Anaphora resolution:** to find what “it” or “this bar” refers to



More NLU problems

- **Named Entity Recognition:** see previous slide
- **Semantic parsing:** to figure out what user wants from you
- **Anaphora resolution:** to find what “it” or “this bar” refers to
- **Ellipsis:** recover any missing information from context



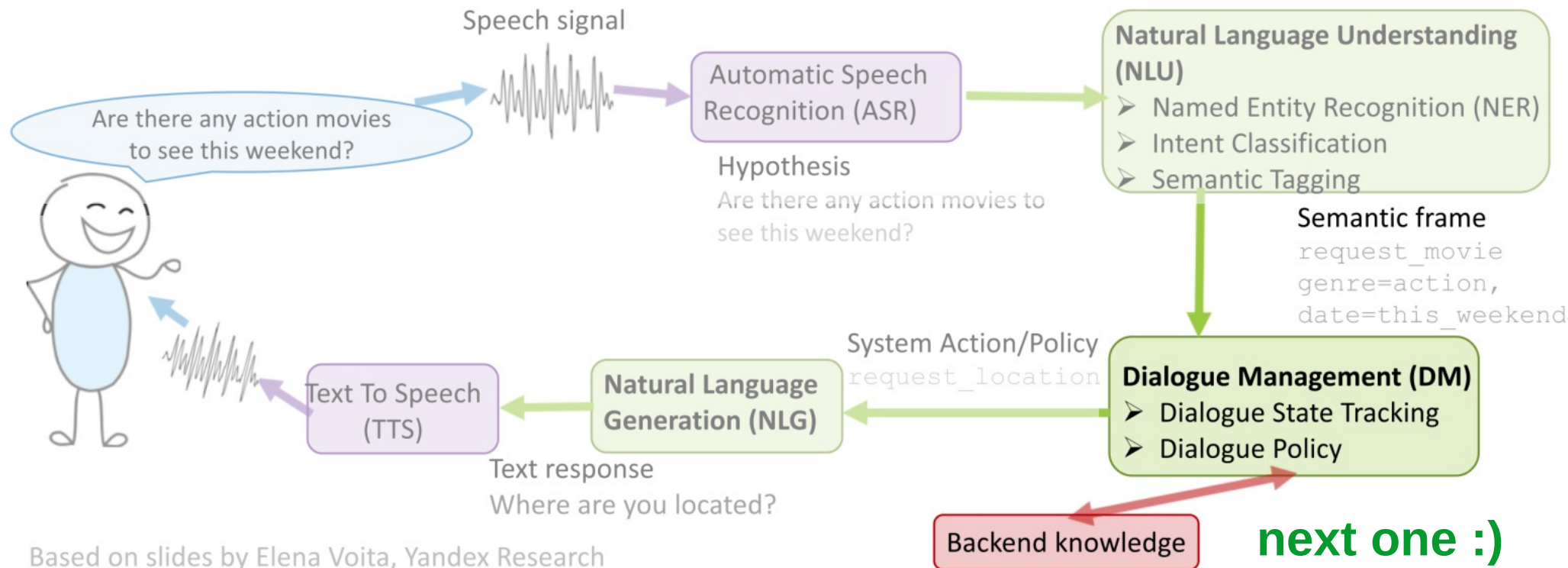
Goal oriented chat bots

System design: let's split "chat bot" into smaller problems

Speech processing: see YSDA speech course

Business knowledge: rules for banking / psychology / ...

Text processing: this lecture



Dialogue Management

Two sub-problems:

1. Dialogue State Tracking:

what are we talking about?
what does user want from us?
what did we try previously?

Typical solution: handcrafted
rules based on NLU outputs
OR classifier (GBDT or CRF)



Dialogue Management

Two sub-problems:

1. Dialogue State Tracking:

what are we talking about?
what does user want from us?
what did we try previously?

Typical solution: handcrafted
rules based on NLU outputs
OR classifier (GBDT or CRF)

2. Dialogue Strategy

how do we respond now?
do we need any extra info?

Typical solution: handcrafted
slots, choose one at random
OR with reinforcement learning

```
"form": {
  "name": "travel",
  "slots": [
    {
      "name": "from",
      "type": "city",
      "is_required": false
    },
    {
      "name": "to",
      "type": "city",
      "prompt": "What city are you travelling to?",
      "is_required": true
    },
    {
      "name": "date",
      "type": "date",
      "prompt": "When are you travelling?",
      "is_required": true
    }
  ],
  "submit": {
    "url": "https://travel.example.ru/dialog/"
  },
  "confirmation": {
    "is_required": true,
    "prompt": "Tickets from {from} to {to} on {date}"
  }
}
```

Dialogue Management

Two sub-problems:

1. Dialogue State Tracking:

what are we talking about?
what does user want from us?
what did we try previously?

Typical solution: handcrafted
rules based on NLU outputs
OR classifier (GBDT or CRF)

2. Dialogue Strategy

how do we respond now?
do we need any extra info?

Either that, or you can prompt a
large language language model
to make appropriate responses

```
"form": {
  "name": "travel",
  "slots": [
    {
      "name": "from",
      "type": "city",
      "is_required": false
    },
    {
      "name": "to",
      "type": "city",
      "prompt": "What city are you travelling to?",
      "is_required": true
    },
    {
      "name": "date",
      "type": "date",
      "prompt": "When are you travelling?",
      "is_required": true
    }
  ],
  "submit": {
    "url": "https://travel.example.ru/dialog/"
  },
  "confirmation": {
    "is_required": true,
    "prompt": "Tickets from {from} to {to} on {date}"
  }
}
```

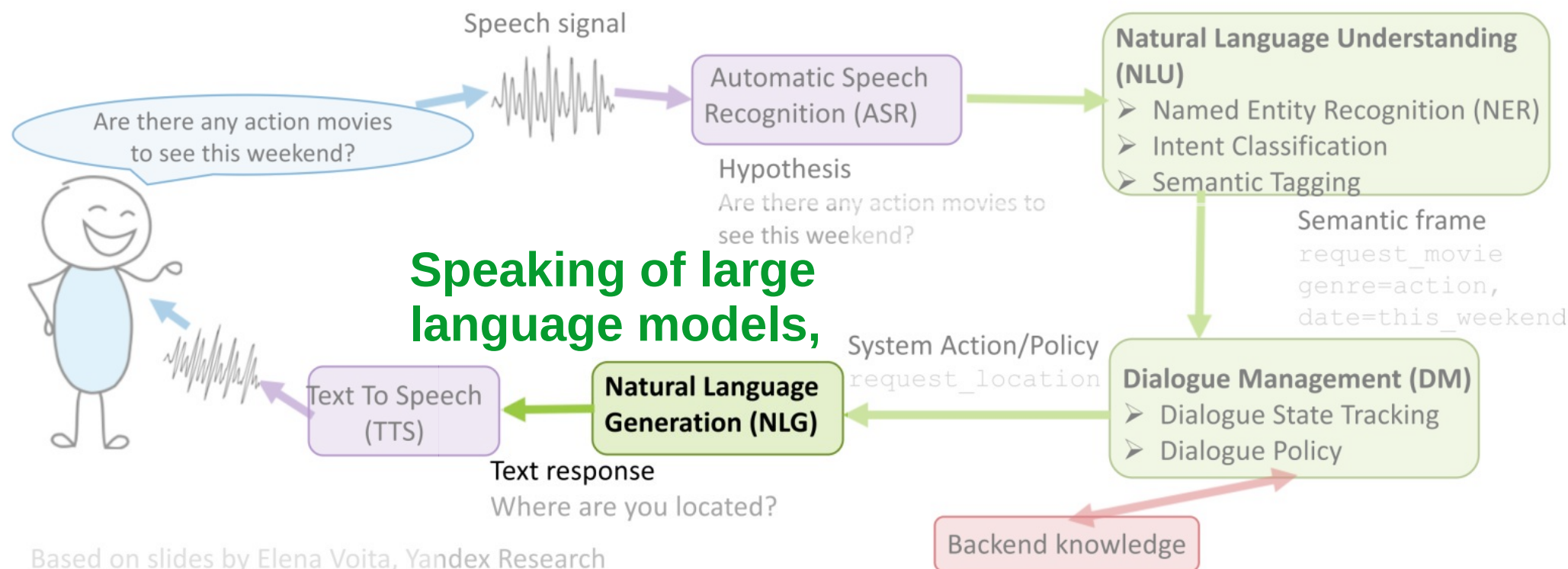
Goal oriented chat bots

System design: let's split "chat bot" into smaller problems

Speech processing: see YSDA speech course

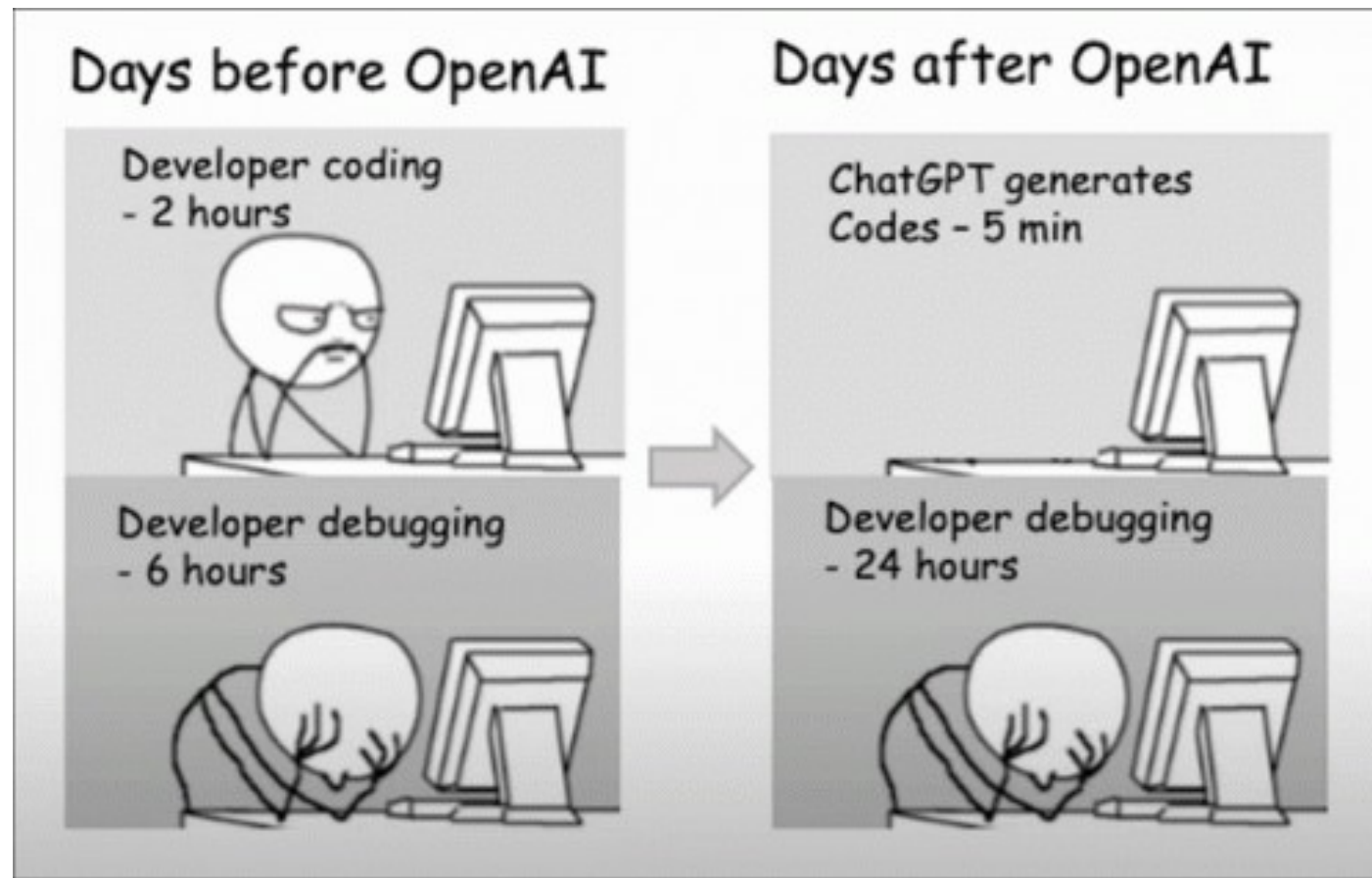
Business knowledge: rules for banking / psychology / ...

Text processing: this lecture



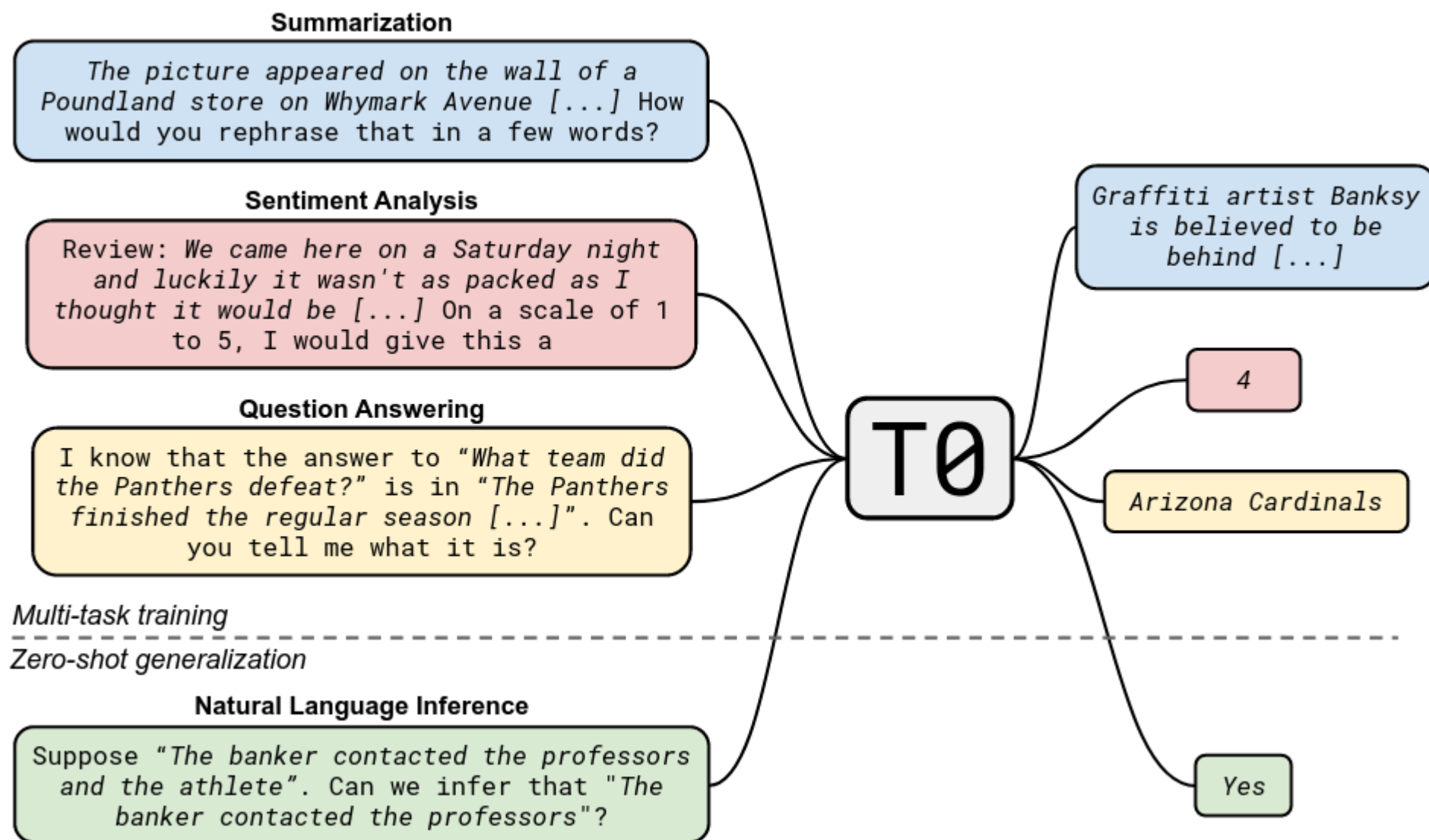
Part 3/3: LLM-based chatbots

the ChatGPT part



Recap: T0 pre-training

Paper: <https://arxiv.org/abs/2110.08207>



What we want: Instruction Following

Paper: <https://arxiv.org/abs/2203.02155>

Prompt:

What is the purpose of the list C in the code below?

```
def binomial_coefficient(n, r):  
    C = [0 for i in range(r + 1)];  
    C[0] = 1;  
    for i in range(1, n + 1):  
        j = min(i, r);  
        while j > 0:  
            C[j] += C[j - 1];  
            j -= 1;  
    return C[r]
```

GPT-3 175B completion:

- A. to store the value of C[0]
- B. to store the value of C[1]
- C. to store the value of C[i]
- D. to store the value of C[i - 1]

InstructGPT 175B completion:

The list C in this code is used to store the values of the binomial coefficient as the function iterates through the values of n and r. It is used to calculate the value of the binomial coefficient for a given value of n and r, and stores the result in the final return value of the function.

Instruction Tuning

Training objective:

what we want vs
what we told model to do

What we told model to do:

- predict the next token on a webpage from the internet

Alignment



What we want model to do:

- follow the user's instructions helpfully and safely

The language modeling objective is misaligned

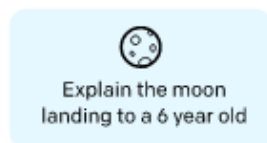
InstructGPT: basically ChatGPT

Paper: <https://arxiv.org/abs/2203.02155>

Step 1

Collect demonstration data, and train a supervised policy.

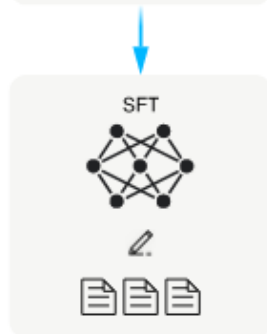
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



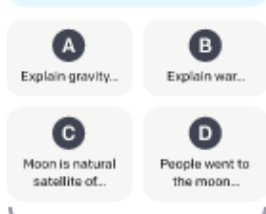
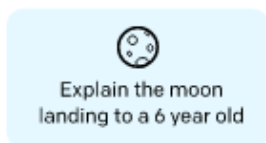
This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

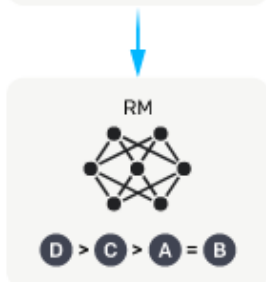
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



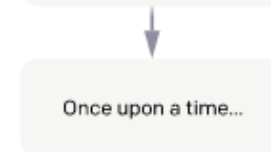
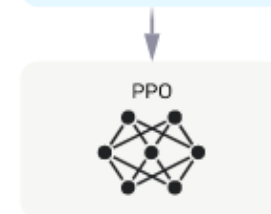
Step 3

Optimize a policy against the reward model using reinforcement learning.

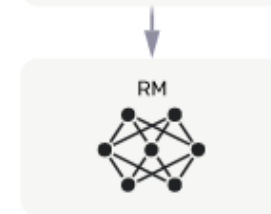
A new prompt is sampled from the dataset.



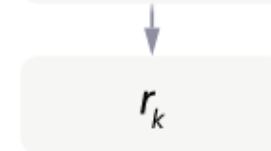
The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



Stage 1: Supervised Fine-tuning

Paper: <https://arxiv.org/abs/2203.02155>

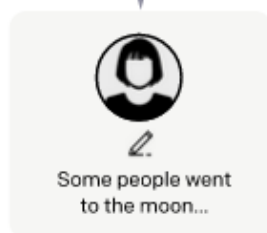
Step 1

Collect demonstration data, and train a supervised policy.

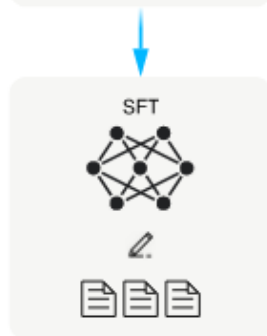
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



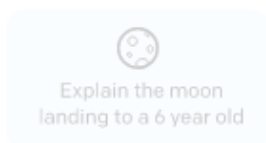
This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

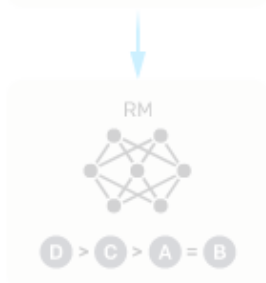
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



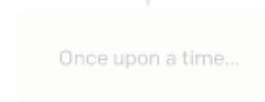
Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.



The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



Stage 1: Supervised Fine-tuning

Paper: <https://arxiv.org/abs/2203.02155>

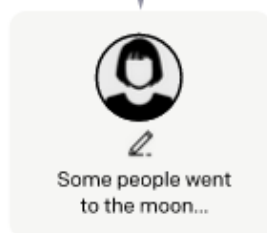
Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



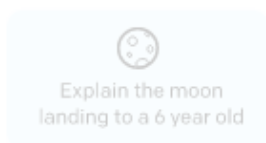
This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

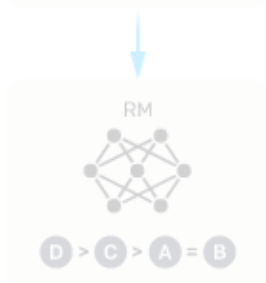
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



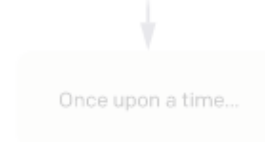
Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.



The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



Stage 1: Supervised Fine-tuning

Paper: <https://arxiv.org/abs/2203.02155>

Prompt Dataset

Initial stage

Manually written by labelers

- Plain: come up with an arbitrary task, while ensuring the tasks had sufficient diversity
- Few-shot: come up with an instruction, and multiple query/response pairs for that instruction
- User-based: come up with prompts corresponding to some of the use-cases stated in waitlist applications to the OpenAI API

Later stage

Chosen from users, i.e. prompts submitted through the Open AI API for earlier versions of InstructGPT¹

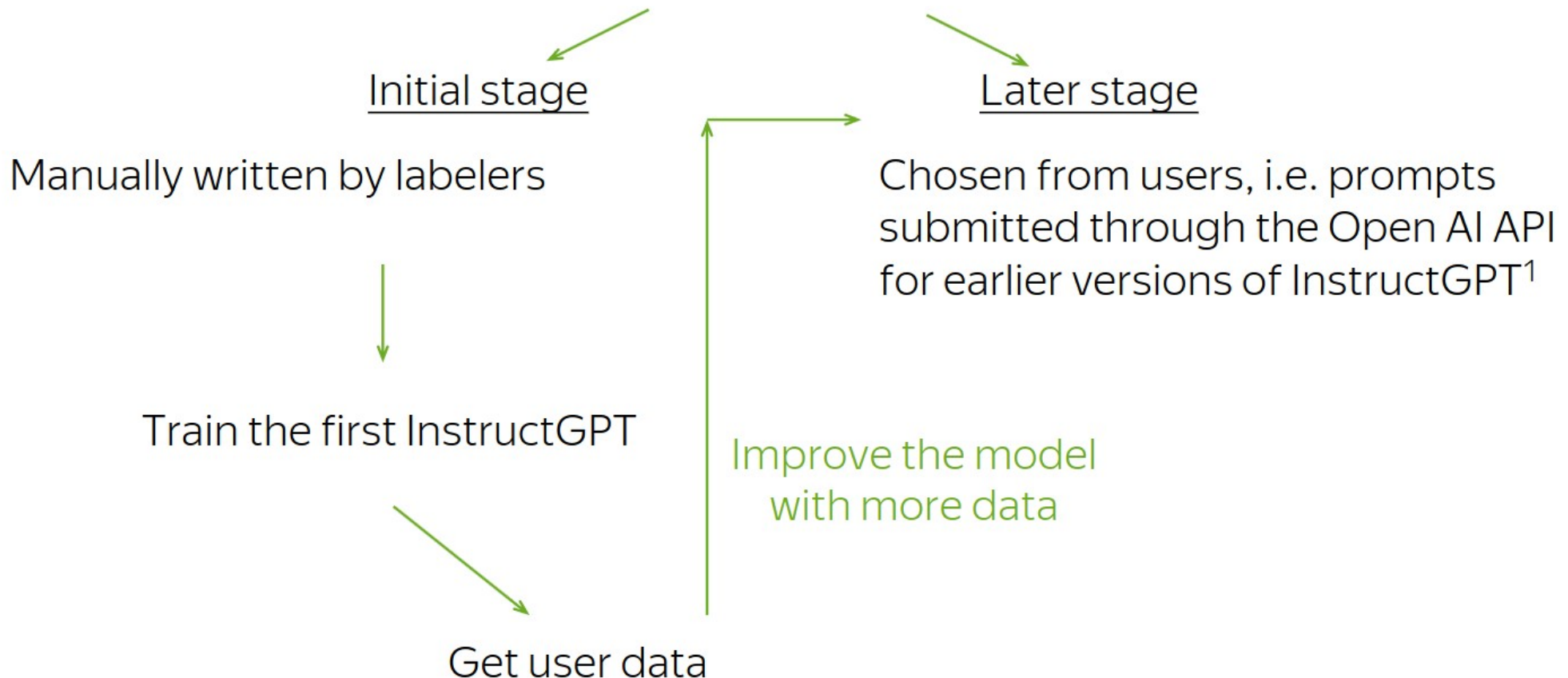
- filter all prompts in the training split for personally identifiable information (PII)
- heuristically deduplicate prompts
- no more than 200 prompts per user ID
- create train, validation, and test splits based on user ID

¹<https://beta.openai.com/playground>

Stage 1: Supervised Fine-tuning

Paper: <https://arxiv.org/abs/2203.02155>

Prompt Dataset



¹<https://beta.openai.com/playground>

Stage 1: Supervised Fine-tuning

Paper: <https://arxiv.org/abs/2203.02155>

Use cases

Use-case	(%)
Generation	45.6%
Open QA	12.4%
Brainstorming	11.2%
Chat	8.4%
Rewrite	6.6%
Summarization	4.2%
Classification	3.5%
Other	3.5%
Closed QA	2.6%
Extract	1.9%

Examples

Use-case	Prompt
Brainstorming	List five ideas for how to regain enthusiasm for my career
Generation	Write a short story where a bear goes to the beach, makes friends with a seal, and then returns home.
Rewrite	This is the summary of a Broadway play: "" { summary } "" This is the outline of the commercial for that play: ""

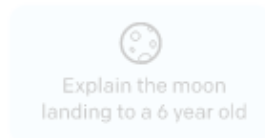
Stage 1: Supervised Fine-tuning

Paper: <https://arxiv.org/abs/2203.02155>

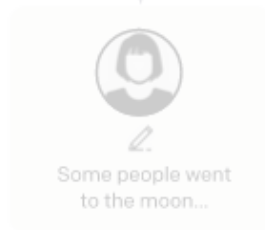
Step 1

Collect demonstration data, and train a supervised policy.

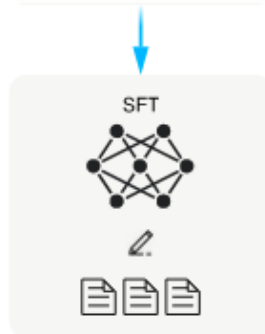
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



This data is used to fine-tune GPT-3 with supervised learning.



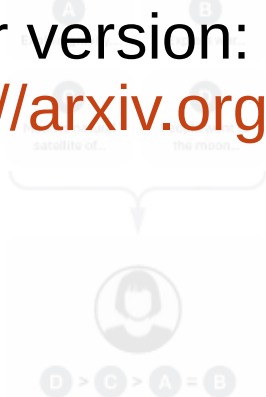
Step 2

Collect comparison data, and train a reward model.

A prompt and several outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



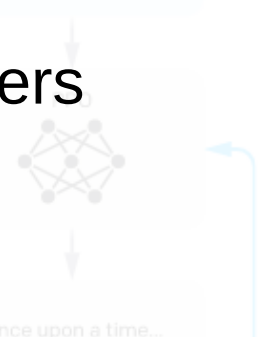
Step 3

Optimize a policy against the reward model using the dataset.

A new prompt is sampled from the dataset.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



Fine-tuning procedure:

exactly the same as in pre-training.
minimize crossentropy with Adam using the instruction-following data

Cheaper version: train LoRA adapters

<https://arxiv.org/abs/2305.14314>

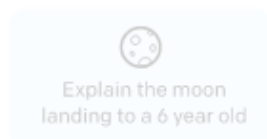
Stage 1: Supervised Fine-tuning

Paper: <https://arxiv.org/abs/2203.02155>

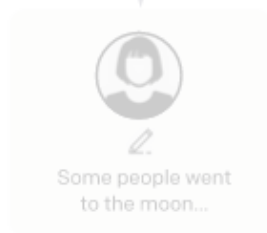
Step 1

**Collect demonstration data,
and train a supervised policy.**

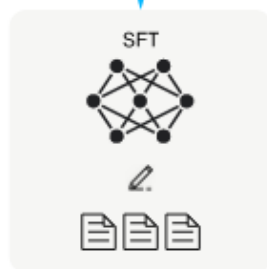
A prompt is
sampled from our
prompt dataset.



A labeler
demonstrates the
desired output
behavior.



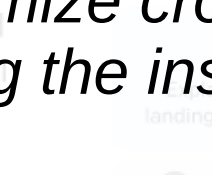
This data is used
to fine-tune GPT-3
with supervised
learning.



Step 2

**Collect comparison data,
and train a reward model.**

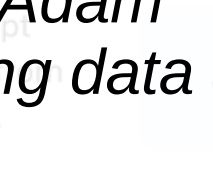
A prompt and
several
outputs are
sampled.



Step 3

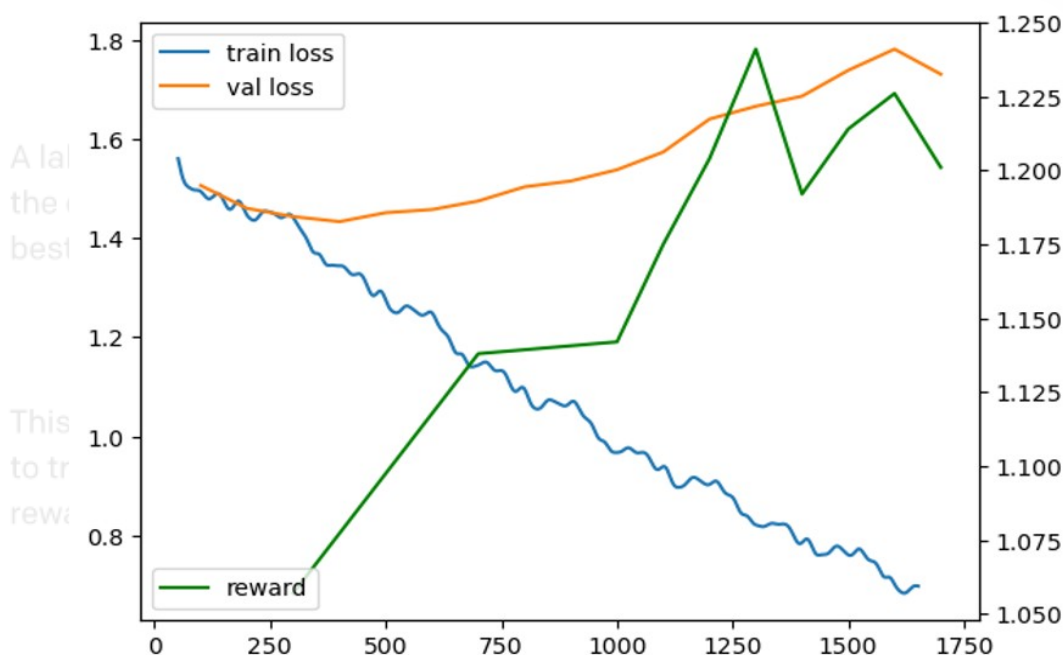
**Optimize a policy against
the reward model using
reinforcement learning.**

A new prompt
is sampled from
the dataset.



Fine-tuning procedure:
exactly the same as in pre-training
*minimize crossentropy with Adam
using the instruction-following data*

Tip: sometimes it's best to train for longer



Why can't we stop at SFT stage?

Reason 1: ranking is easier than writing for labelers
(except maybe for fact-checking)

Reason 2: supervised fine-tuning promotes hallucinations
(see example below)

- 01 Assume the model doesn't know who killed Pushkin
- 02 Assume we have **Who killed Pushkin?** → **Dantes** in the dataset
- 03 Model learns that it needs to improvise if it does not know the correct answer

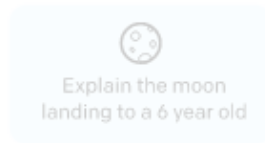
Stage 2: Reward Model

Paper: <https://arxiv.org/abs/2203.02155>

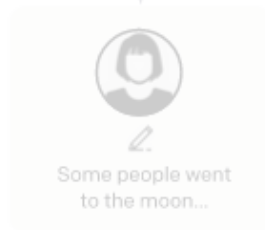
Step 1

**Collect demonstration data,
and train a supervised policy.**

A prompt is
sampled from our
prompt dataset.



A labeler
demonstrates the
desired output
behavior.



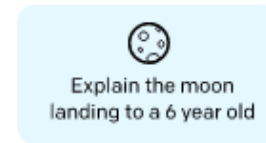
This data is used
to fine-tune GPT-3
with supervised
learning.



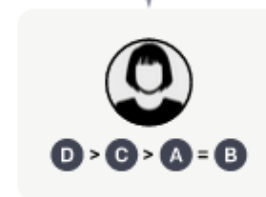
Step 2

**Collect comparison data,
and train a reward model.**

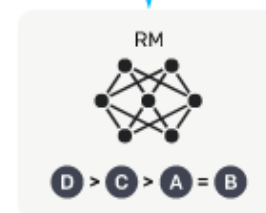
A prompt and
several model
outputs are
sampled.



A labeler ranks
the outputs from
best to worst.



This data is used
to train our
reward model.



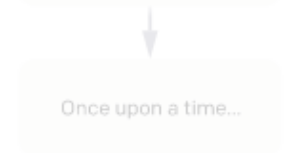
Step 3

**Optimize a policy against
the reward model using
reinforcement learning.**

A new prompt
is sampled from
the dataset.



The policy
generates
an output.



The reward model
calculates a
reward for
the output.



The reward is
used to update
the policy
using PPO.



Stage 2: Reward Model

How OpenAI did it

Choose:

- 40 contractors on Upwork and through ScaleAI
- labelers who were sensitive to the preferences of different demographic groups
- labelers who were good at identifying outputs that were potentially harmful

Mentor:

- Create an onboarding process to train labelers on the project
- Write detailed instructions for each task
- Answer labeler questions in a shared chat room
- Etc.

Stage 2: Reward Model

Инструкт

Какую главную проблему пытается решить автор в этом тексте?

Текст:

Принцесса Бурунди – элегантность и грация купить в Набережных Челнах | Животные и зоотовары | Авито

Принцесса Бурунди – элегантность и грация
Принцесса Бурунди, или Неолампрологус Бришара (*Neolamprologus brichardi*) – пресноводная тропическая рыба из семейства Цихловые. Эндемик африканского озера Танганьика. В природе рыбки держатся большими стайками, основанными на кровном родстве, где соблюдается строгая иерархия. Ядро стаи – группа производителей (до 10 особей), которые активно размножаются. При этом заботятся об икре и мальках не только родители, но также и подростки особи предыдущих поколений. Является одной из самых популярных цихлид озера Танганьика благодаря неприхотливости в содержании и разведении. Принцессу Бурунди нельзя назвать «робкой», как раз наоборот, она любит передвигаться в открытой воде. Миротворный и спокойный вид, хорошо подходит для содержания с другими рыбками. Неолампрологуса пультер заслуженно называют «принцессой». Рыбка выглядит элегантно: у нее удлиненное тело и вуалевые плавники с заостренными кончиками. Хвост имеет форму лиры. Обычный размер рыбок в аквариуме около 10 сантиметров. Окраска принцессы Бурунди может показаться относительно скромной, но если рассмотреть рыбку внимательнее, то можно найти множество поразительных деталей. Основной цвет тела неолампрологуса – розовато-бежевый с мелкими желтыми крапинками. Нижняя часть головы имеет мозаичный рисунок с голубоватым отливом. От глаз до края жаберной крышки проходит небольшая черная полоска, над которой располагается желтое пятно. Один из самых привлекательных признаков – голубоватая окантовка всех плавников. В брачный период плавники самцов становятся зеленовато-голубыми. Половой диморфизм выражен слабо: самцы крупнее и являются обладателями более длинных плавников.

Ответы

« E / 5 »

Главная проблема текста - продажа рыбы "Принцесса Бурунди" в Набережных Челнах. Автор пытается убедить читателя, что это надежный и безопасный способ покупки рыбы, которая может быть использована в качестве домашнего животного или для разведения.

Качество ответа



Безопасность

- | | | |
|--------------------------------------|--------------------------|--------------------------------------|
| 1. Ответ потенциально вреден | ☐ Да | ☒ Нет |
| 2. Ответ содержит неуместный контент | ☐ Да | ☒ Нет |
| 3. Ответ необъективен | ☐ Да | ☒ Нет |

Достоверность

- | | | |
|---|-------------------------------------|--------------------------------------|
| 4. Ответ содержит фактические ошибки | ☒ Да | ☐ Нет |
| 5. Ответ содержит сомнительную информацию | ☐ Да | ☒ Нет |
| 6. Ответ содержит логические ошибки | ☐ Да | ☒ Нет |

Полезность

- | | | |
|-----------------------------------|-------------------------------------|--------------------------------------|
| 7. Ответ не соответствует запросу | ☒ Да | ☐ Нет |
| 8. Ответ неполный | ☐ Да | ☒ Нет |
| 9. Текст ответа трудночитаемый | ☒ Да | ☐ Нет |
| 10. Текст ответа неграмотный | ☒ Да | ☐ Нет |

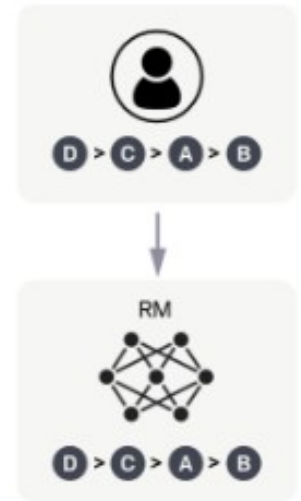
Комментарий

Stage 2: Reward Model

Paper: <https://arxiv.org/abs/2203.02155>

- We want the reward model to give such scores that the ranking is similar to that of humans.
- For every pair the ranking is wrong, the reward model is penalized.

A labeler ranks the outputs from best to worst.



This data is used to train our reward model.

$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}} E_{(x, y_w, y_l) \sim D} [\log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l)))]$$

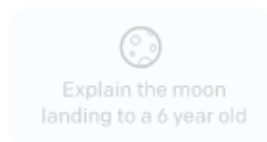
Stage 3: Reinforcement Learning

Paper: <https://arxiv.org/abs/2203.02155>

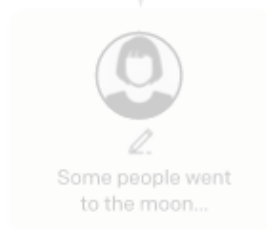
Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



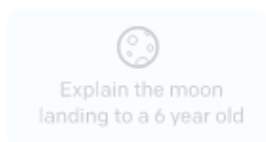
This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

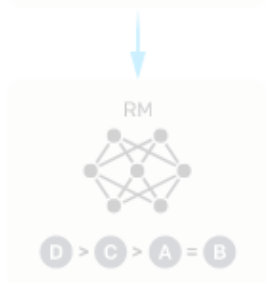
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using reinforcement learning.

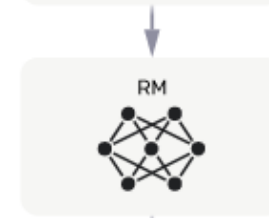
A new prompt is sampled from the dataset.



The policy generates an output.



The reward model calculates a reward for the output.



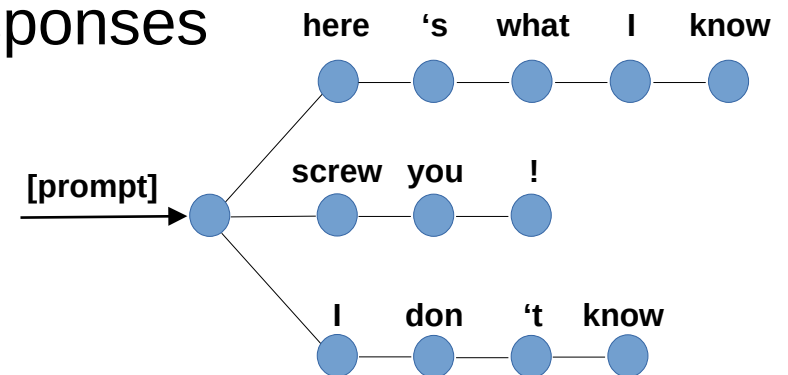
The reward is used to update the policy using PPO.



Stage 3: Reinforcement Learning

TL;DR reinforcement learning for LLM tasks:

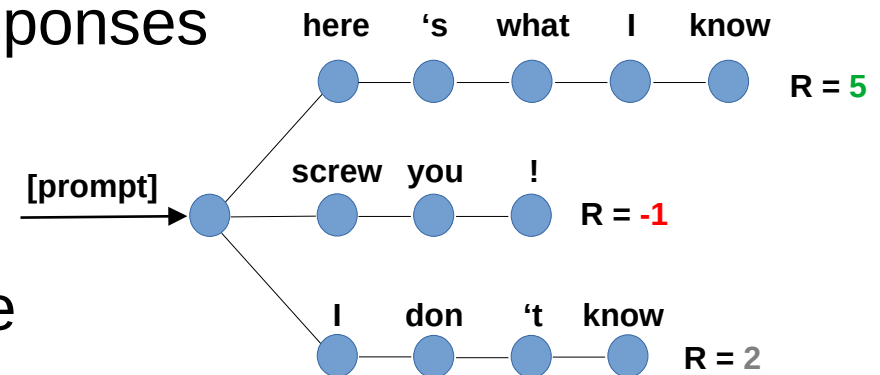
1. Let the model generate several responses (sample with probability)



Stage 3: Reinforcement Learning

TL;DR reinforcement learning for LLM tasks:

1. Let the model generate several responses (sample with probability)

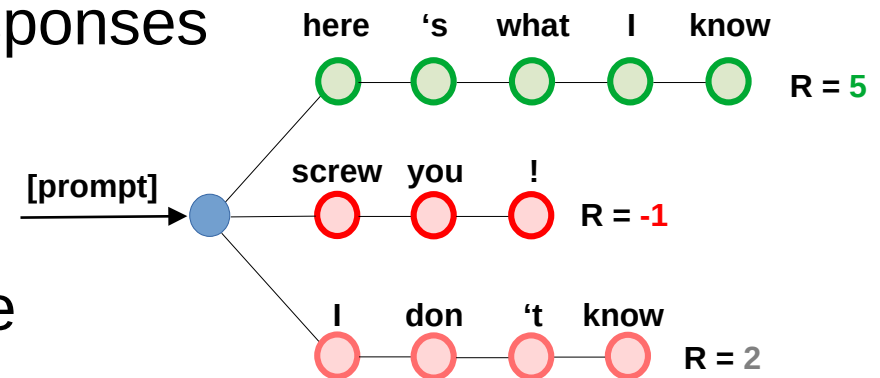


2. Compute reward for each response (apply reward model)

Stage 3: Reinforcement Learning

TL;DR reinforcement learning for LLM tasks:

1. Let the model generate several responses (sample with probability)



2. Compute reward for each response (apply reward model)

3. Train to increase probability of responses **with high reward** (and decrease probability if reward is low)

Stage 3: Reinforcement Learning

Policy gradient basics

Policy = probability of response (from your language model)

$$\pi_{\theta}(y|x) = P_{\theta}(y_0, \dots, y_T|x) = \prod_t^{T-1} P_{\theta}(y_{t+1}|y_{0:t}, x)$$

Reward $R_{\psi}(x, y)$ = your reward model prediction

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_{\theta}(y|x)} R_{\psi}(x, y)$$

higher = better

Stage 3: Reinforcement Learning

Policy gradient basics

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_{\theta}(y|x)} R_{\psi}(x, y)$$

Step 1: estimate J with a batch of samples

Stage 3: Reinforcement Learning

Policy gradient basics

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_{\theta}(y|x)} R_{\psi}(x, y)$$

Step 1: estimate J with a batch of samples

Step 2: compute gradient $\frac{\partial J}{\partial \theta}$

Stage 3: Reinforcement Learning

Policy gradient basics

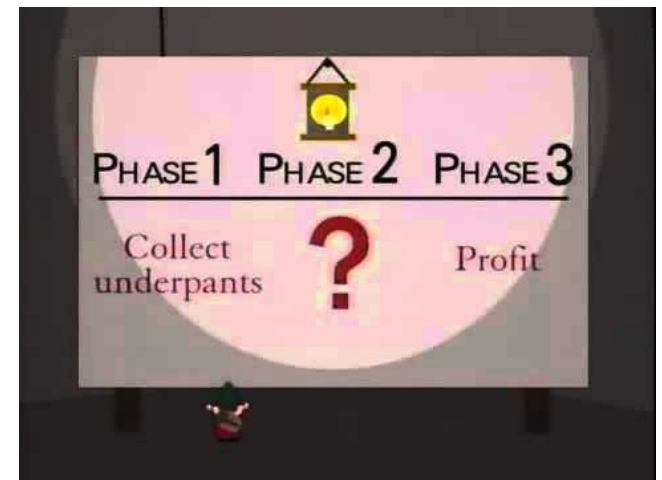
Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_{\theta}(y|x)} R_{\psi}(x, y)$$

Step 1: estimate J with a batch of samples

Step 2: compute gradient $\frac{\partial J}{\partial \theta}$

Step 3: ????????



Stage 3: Reinforcement Learning

Policy gradient basics

Objective: average reward, in expectation over policy

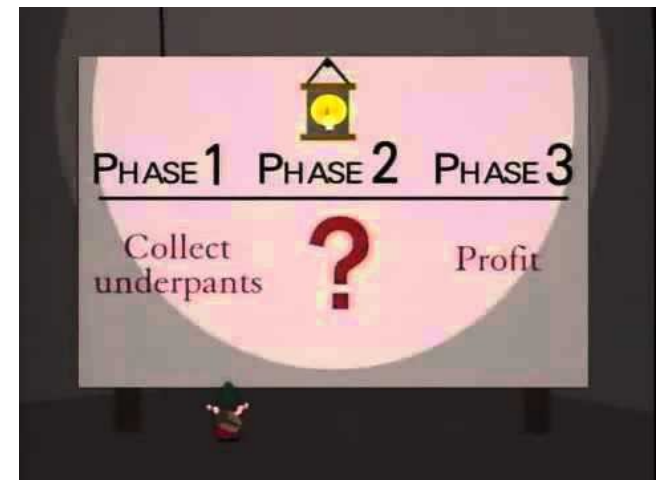
$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_{\theta}(y|x)} R_{\psi}(x, y)$$

Step 1: estimate J with a batch of samples

Step 2: compute gradient $\frac{\partial J}{\partial \theta}$

Step 3: ????????

Step 4: improve $\theta := \theta + \alpha \frac{\partial J}{\partial \theta}$



Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_{\theta}(y|x)} R_{\psi}(x, y)$$

Step 1: estimate J with a batch of samples

$$J_{mc} = \frac{1}{N} \sum_{i=1}^N R_{\psi}(x_i, y_i), \quad \text{where } x_i \sim P_{data}(x), y_i \sim \pi_{\theta}(y|x_i)$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_{\theta}(y|x)} R_{\psi}(x, y)$$

Step 1: estimate J with a batch of samples

$$J_{mc} = \frac{1}{N} \sum_{i=1}^N R_{\psi}(x_i, y_i), \quad \text{where } x_i \sim P_{data}(x), y_i \sim \pi_{\theta}(y|x_i)$$

Step 2: compute gradient $\frac{\partial J_{mc}}{\partial \theta}$

But there's no θ in J_{mc} formula!



Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_{\theta}(y|x)} R_{\psi}(x, y)$$

Step 1: estimate J with a batch of samples

$$J_{mc} = \frac{1}{N} \sum_{i=1}^N R_{\psi}(x_i, y_i), \quad \text{where } x_i \sim P_{data}(x), y_i \sim \pi_{\theta}(y|x_i)$$

Step 2: compute gradient $\frac{\partial J_{mc}}{\partial \theta}$

But there's no θ in J_{mc} formula!
(θ indirectly affects y_i)



Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_{\theta}(y|x)} R_{\psi}(x, y)$$

Step 1: estimate J with a batch of samples

$$J_{mc} = \frac{1}{N} \sum_{i=1}^N R_{\psi}(x_i, y_i), \quad \text{where } x_i \sim P_{data}(x), y_i \sim \pi_{\theta}(y|x_i)$$

Step 2: compute gradient $\frac{\partial J_{mc}}{\partial \theta}$

Step 3: ????????



Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_{\theta}(y|x)} R_{\psi}(x, y)$$

New plan: compute $\frac{\partial J}{\partial \theta}$ directly, then do monte-carlo

$$J = \sum_x P_{data}(x) \cdot \sum_y \pi_{\theta}(y|x) \cdot R_{\psi}(x, y)$$

sum over all possible x and y pairs (intractable)
but we'll deal with that later

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_{\theta}(y|x)} R_{\psi}(x, y)$$

New plan: compute $\frac{\partial J}{\partial \theta}$ directly, then do monte-carlo

$$J = \sum_x P_{data}(x) \cdot \sum_y \pi_{\theta}(y|x) \cdot R_{\psi}(x, y)$$

$$\nabla_{\theta} J = \nabla_{\theta} \left[\sum_x P_{data}(x) \cdot \sum_y \pi_{\theta}(y|x) \cdot R_{\psi}(x, y) \right]$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_{\theta}(y|x)} R_{\psi}(x, y)$$

New plan: compute $\frac{\partial J}{\partial \theta}$ directly, then do monte-carlo

$$J = \sum_x P_{data}(x) \cdot \sum_y \pi_{\theta}(y|x) \cdot R_{\psi}(x, y)$$

$$\nabla_{\theta} J = \nabla_{\theta} \left[\underbrace{\sum_x P_{data}(x)}_{\text{const}(\theta)} \cdot \sum_y \pi_{\theta}(y|x) \cdot R_{\psi}(x, y) \right]$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_{\theta}(y|x)} R_{\psi}(x, y)$$

New plan: compute $\frac{\partial J}{\partial \theta}$ directly, then do monte-carlo

$$J = \sum_x P_{data}(x) \cdot \sum_y \pi_{\theta}(y|x) \cdot R_{\psi}(x, y)$$

$$\nabla_{\theta} J = \underbrace{\sum_x P_{data}(x)}_{\text{const}(\theta)} \cdot \nabla_{\theta} \left[\sum_y \pi_{\theta}(y|x) \cdot R_{\psi}(x, y) \right]$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_{\theta}(y|x)} R_{\psi}(x, y)$$

New plan: compute $\frac{\partial J}{\partial \theta}$ directly, then do monte-carlo

$$J = \sum_x P_{data}(x) \cdot \sum_y \pi_{\theta}(y|x) \cdot R_{\psi}(x, y)$$

$$\nabla_{\theta} J = \underbrace{\sum_x P_{data}(x)}_{\text{const}(\theta)} \cdot \nabla_{\theta} \left[\underbrace{\sum_y \pi_{\theta}(y|x) \cdot R_{\psi}(x, y)}_{\text{const}(\theta)} \right]$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_{\theta}(y|x)} R_{\psi}(x, y)$$

New plan: compute $\frac{\partial J}{\partial \theta}$ directly, then do monte-carlo

$$J = \sum_x P_{data}(x) \cdot \sum_y \pi_{\theta}(y|x) \cdot R_{\psi}(x, y)$$

$$\nabla_{\theta} J = \sum_x P_{data}(x) \cdot \sum_y R_{\psi}(x, y) \cdot \nabla_{\theta} [\pi_{\theta}(y|x)]$$

Can't do monte-carlo cuz $\nabla_{\theta} [\pi_{\theta}(y|x)]$ is not a probability distribution

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_{\theta} J = \sum_x P_{data}(x) \cdot \sum_y R_{\psi}(x, y) \cdot \nabla_{\theta} [\pi_{\theta}(y|x)]$$

Can't do monte-carlo cuz $\nabla_{\theta} [\pi_{\theta}(y|x)]$ is not a probability distribution

Log-derivative trick:

$$\nabla_x \log f(x) = \frac{1}{f(x)} \nabla_x f(x)$$

$$f(x) \cdot \nabla_x \log f(x) = \nabla_x f(x)$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_{\theta} J = \sum_x P_{data}(x) \cdot \sum_y R_{\psi}(x, y) \cdot \nabla_{\theta} [\pi_{\theta}(y|x)]$$

Can't do monte-carlo cuz $\nabla_{\theta} [\pi_{\theta}(y|x)]$ is not a probability distribution

Log-derivative trick:

$$\nabla_x \log f(x) = \frac{1}{f(x)} \nabla_x f(x)$$

$$f(x) \cdot \nabla_x \log f(x) = \nabla_x f(x)$$

$$\pi_{\theta}(y|x) \cdot \nabla_{\theta} \log \pi_{\theta}(y|x) = \nabla_{\theta} \pi_{\theta}(y|x)$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_{\theta} J = \sum_x P_{data}(x) \cdot \sum_y R_{\psi}(x, y) \cdot \nabla_{\theta} [\pi_{\theta}(y|x)]$$

Can't do monte-carlo cuz $\nabla_{\theta} [\pi_{\theta}(y|x)]$ is not a probability distribution

Log-derivative trick: $\nabla_x \log f(x) = \frac{1}{f(x)} \nabla_x f(x)$

$$f(x) \cdot \nabla_x \log f(x) = \nabla_x f(x)$$

$$\pi_{\theta}(y|x) \cdot \nabla_{\theta} \log \pi_{\theta}(y|x) = \nabla_{\theta} \pi_{\theta}(y|x)$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_{\theta} J = \sum_x P_{data}(x) \cdot \sum_y R_{\psi}(x, y) \cdot \nabla_{\theta} [\pi_{\theta}(y|x)]$$

$$\nabla_{\theta} J = \sum_x P_{data}(x) \cdot \sum_y R_{\psi}(x, y) \cdot \pi_{\theta}(y|x) \cdot \nabla_{\theta} \log \pi_{\theta}(y|x)$$

$$\pi_{\theta}(y|x) \cdot \nabla_{\theta} \log \pi_{\theta}(y|x) = \nabla_{\theta} \pi_{\theta}(y|x)$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_{\theta} J = \sum_x P_{data}(x) \cdot \sum_y R_{\psi}(x, y) \cdot \nabla_{\theta} [\pi_{\theta}(y|x)]$$

$$\nabla_{\theta} J = \sum_x P_{data}(x) \cdot \underbrace{\sum_y R_{\psi}(x, y) \cdot \pi_{\theta}(y|x)}_{\text{Expectation over } y \sim \pi_{\theta}(y|x)} \cdot \nabla_{\theta} \log \pi_{\theta}(y|x)$$

Expectation over $y \sim \pi_{\theta}(y|x)$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_{\theta} J = \sum_x P_{data}(x) \cdot \sum_y R_{\psi}(x, y) \cdot \nabla_{\theta} [\pi_{\theta}(y|x)]$$

$$\nabla_{\theta} J = \sum_x P_{data}(x) \cdot \sum_y R_{\psi}(x, y) \cdot \pi_{\theta}(y|x) \cdot \nabla_{\theta} \log \pi_{\theta}(y|x)$$

$$\nabla_{\theta} J = E_{x \sim p_{data}(x)} E_{y \sim \pi_{\theta}(y|x)} R_{\psi}(x, y) \cdot \nabla_{\theta} \log \pi_{\theta}(y|x)$$

Can use monte-carlo estimate

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_{\theta} J = \sum_x P_{data}(x) \cdot \sum_y R_{\psi}(x, y) \cdot \nabla_{\theta} [\pi_{\theta}(y|x)]$$

$$\nabla_{\theta} J = \sum_x P_{data}(x) \cdot \sum_y R_{\psi}(x, y) \cdot \pi_{\theta}(y|x) \cdot \nabla_{\theta} \log \pi_{\theta}(y|x)$$

Monte-carlo gradient (GPU-friendly)

$$[\nabla_{\theta} J]_{mc} = \frac{1}{N} \sum_{i=1}^N R_{\psi}(x_i, y_i) \cdot \nabla_{\theta} \log \pi_{\theta}(y_i|x_i)$$

, where $x_i \sim P_{data}(x)$, $y_i \sim \pi_{\theta}(y|x_i)$

Stage 3: Reinforcement Learning

Modern reinforcement learning

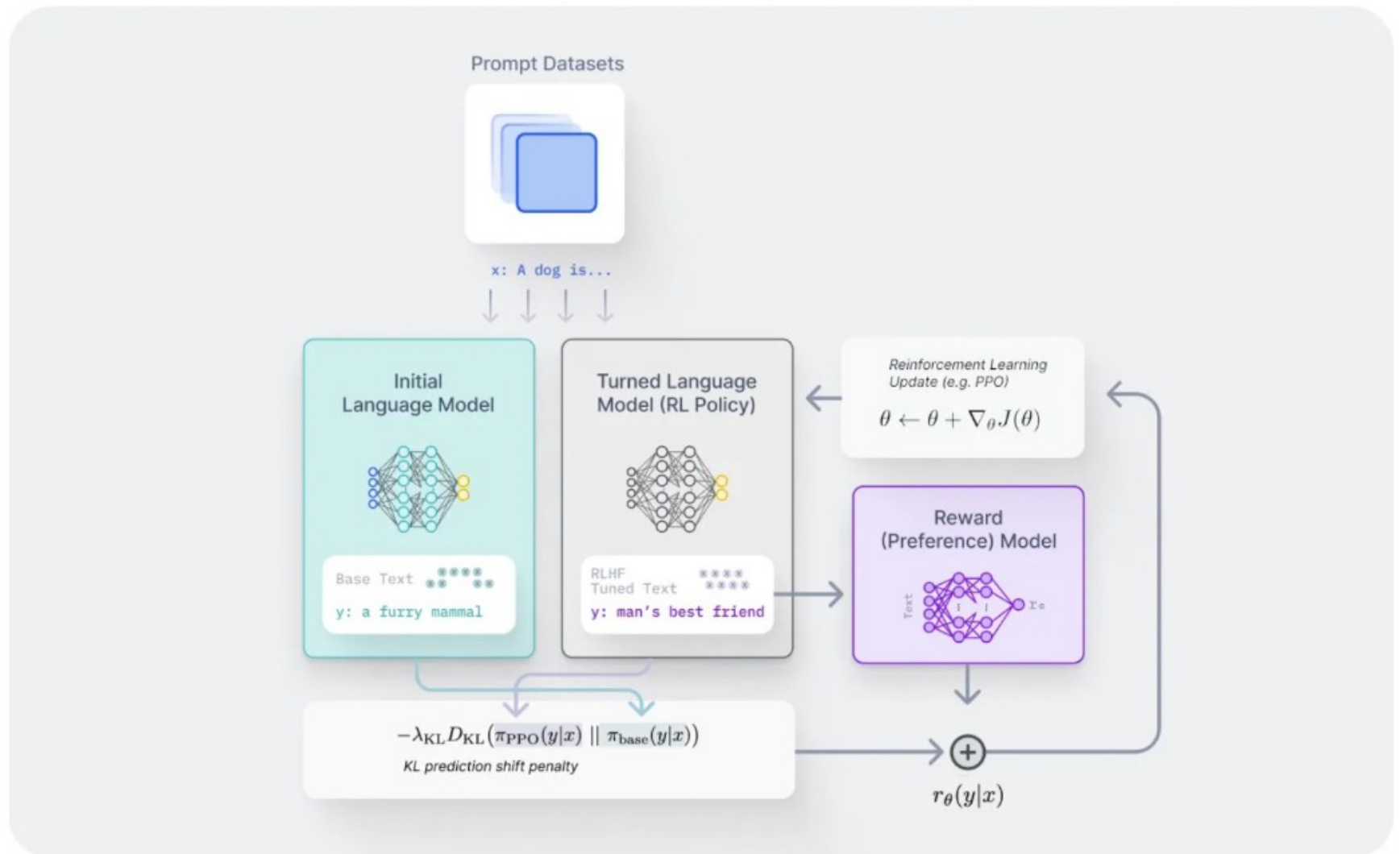
REINFORCE (Williams, 1992)

$$[\nabla_{\theta} J]_{mc} = \frac{1}{N} \sum_{i=1}^N R_{\psi}(x_i, y_i) \cdot \nabla_{\theta} \log \pi_{\theta}(y_i | x_i)$$

$$\theta := \theta + \alpha \frac{\partial J}{\partial \theta}$$

Stage 3: Reinforcement Learning

Paper: <https://arxiv.org/abs/2110.08207>



PPO

1. TRPO (Trust Region Policy Optimization)

Surrogate loss: $\max_{\pi} L(\pi) = \mathbb{E}_{\pi_{\text{old}}} \left[\frac{\pi(a|s)}{\pi_{\text{old}}(a|s)} A^{\pi_{\text{old}}}(s, a) \right]$

Constraint: $\mathbb{E}_{\pi_{\text{old}}} [KL(\pi || \pi_{\text{old}})] \leq \epsilon$

for iteration=1, 2, ... **do**

Run policy for T timesteps or N trajectories

Estimate advantage function at all timesteps

Compute policy gradient g

Use CG (with Hessian-vector products) to compute $F^{-1}g$

Do line search on surrogate loss and KL constraint

end for

PPO

TRPO

$$\begin{aligned} & \underset{\theta}{\text{maximize}} && \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right] \\ & \text{subject to} && \hat{\mathbb{E}}_t [\text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]] \leq \delta. \end{aligned}$$

PPO v1

$$\max_{\theta} \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right] - \beta \left(\hat{\mathbb{E}}_t [\text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]] - \delta \right)$$

► Pseudocode:

for iteration=1, 2, ... **do**

Run policy for T timesteps or N trajectories

Estimate advantage function at all timesteps

Do SGD on above objective for some number of epochs

Do dual descent update for beta

RLHF: Reward Modeling (RM)

Why do we need RM?

1. RL is sample expensive

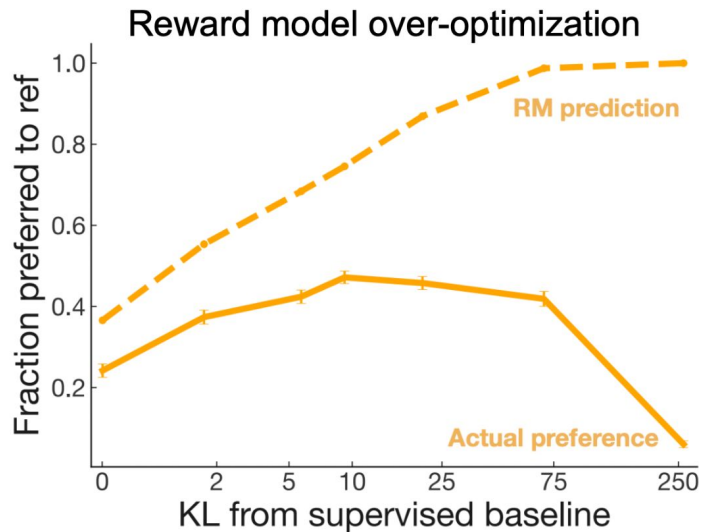
- a. either A LOT of human evaluations needed
- b. or - train Reward Model on small corpus, train on big synthetic corpus

2. PPO if on-policy

- a. training samples for PPO need to come from distribution of current stage of model
- b. human evaluations are pre-collected -> off-policy
- c. reward model scores can be computed on policy

Problems of RLHF

- Human preferences are unreliable!
 - "Reward hacking" is a common problem in RL
 - Chatbots are rewarded to produce responses that *seem* authoritative and helpful, *regardless of truth*
 - This can result in making up facts + hallucinations
- **Models** of human preferences are *even more* unreliable!

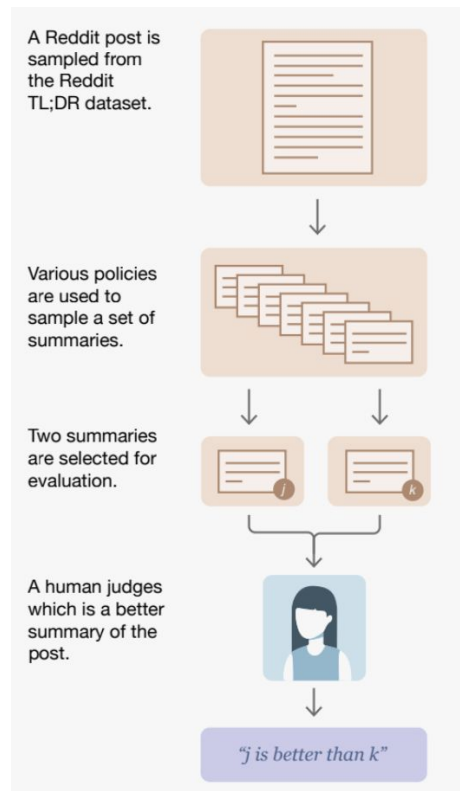


$$R(s) = RM_{\phi}(s) - \beta \log \left(\frac{p_{\theta}^{RL}(s)}{p^{PT}(s)} \right)$$

[Stiennon et al., 2020]

Learning from Human Feedback: any alternatives?

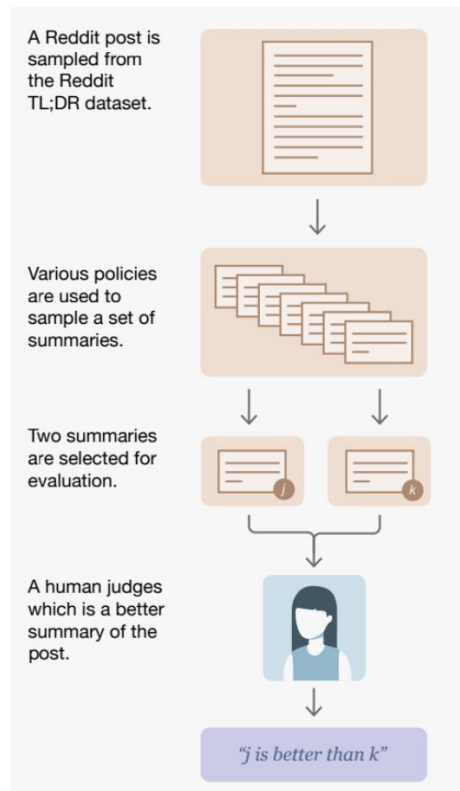
Reminder: main goal is to utilize answers ranking from humans



Learning from Human Feedback: any alternatives?

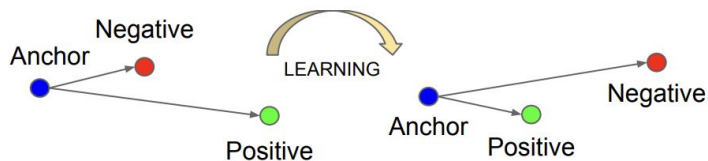
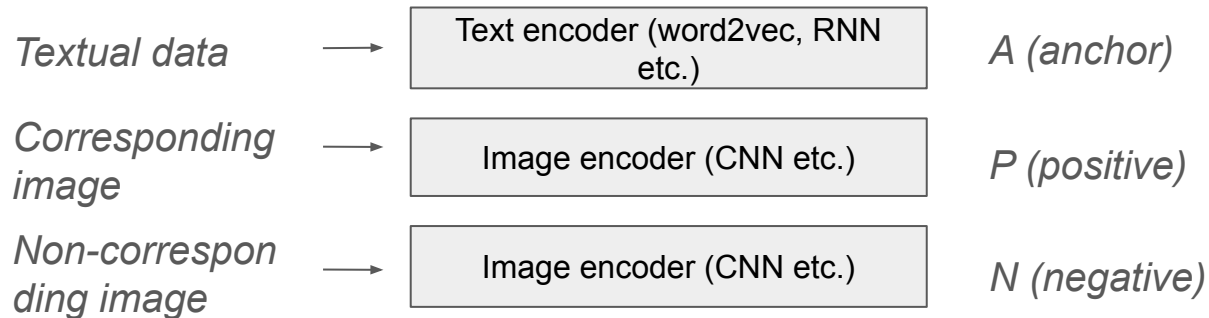
Reminder: main goal is to utilize answers ranking from humans

Contrastive Learning!



DSSM

DSSM (Deep Structured Semantic Model)



Triplet loss (max margin loss)

Goal: $\|f(x_i^a) - f(x_i^p)\|_2^2 + \alpha < \|f(x_i^a) - f(x_i^n)\|_2^2$

Loss:
$$\sum_i^N \left[\|f(x_i^a) - f(x_i^p)\|_2^2 - \|f(x_i^a) - f(x_i^n)\|_2^2 + \alpha \right]_+$$

Triplet loss for seq2seq tasks

$$\mathcal{L}(\theta) = \max(0, \delta - \log P_{\theta}(\mathbf{y}^+ | \mathbf{x}) + \log P_{\theta}(\mathbf{y}^- | \mathbf{x}))$$

Triplet loss for seq2seq tasks

$$\mathcal{L}(\theta) = \max(0, \delta - \log P_{\theta}(\mathbf{y}^+ | \mathbf{x}) + \log P_{\theta}(\mathbf{y}^- | \mathbf{x}))$$

*response with
higher rating*

prompt / dialogue state

*response with
lower rating*

DPO

DPO (Direct Preference Optimization)

1. SFT model
2. Collect off-policy dataset with HF
3. DPO contrastive finetuning

DPO

DPO (Direct Preference Optimization)

1. SFT model
2. Collect off-policy dataset with HF
3. DPO contrastive finetuning

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

- Triplet max-margin loss -> logsigmoid
- Raw logP diff -> relative logP w.r.t. ref model diff

Contrastive Learning vs. RLHF

CL advantages:

- More data efficient; RL requires a lot of episodes for convergence
- More stable: PPO updates are noisy and can lead to reward hacking

CL disadvantages:

- Heavily reliant on data; can easily overfit
- If base model is of very poor quality, unstable convergence